

Mot clé	Description	cas d'utilisation
def	utilisé pour déclarer une variable	Given <code>def var = 0</code>
*	utilisé avant def pour la déclaration de variable elle peut remplacer le Given, When, Then	* <code>def var = 'toto'</code>
print	afficher une variable dans la console	Given <code>def var = 'test'</code> Then <code>print var</code>
text	par défaut Karate parse les variable comme du JSON le mot clé text c'est pour désactiver ce fonctionnement et traiter une variable (reponse...) comme une chaîne text remplace def	Given <code>text = {"name":"toto"}</code> * <code>text = {"name":"toto"}</code>
table	Comme expliqué dans la ligne précédente le JSON est le langage natif de Karate Afin de profiter des mêmes fonctionnalités de Cucumber le mot clé table permet de déclarer un JSON en forme de tableau Karate formate automatiquement ce tableau de paramètres en JSON	Given <code>table clientInfo</code> <code> name lastname </code> <code> dupond Toto </code> <i>Donc Karate traite ce tableau comme</i> <code>clientInfo = '{"name":"dupond", "lastname":"Toto"}</code>
xml	convertir une chaîne de caractère en XML	def <code>strVar =</code> <code>'<root><foo>bar</foo></root>'</code> * <code>xml xmlVar = strVar</code>
url	l'url à appeler par le service (ça peut être l'url du serveur, ou l'url complète y compris les paramètres). cette url reste la même dans le test jusqu'à nouvelle utilisation du mot url une nouvelle fois l'url doit être déclarée en "	Given <code>url</code> <code>'https://jsonplaceholder.typicode.com/</code> <code>'</code> And <code>path</code> <code>'pages/editpage.action?pageId=22649888'</code>

path	path concatène un paramètre REST avec l'url c'est un chemin relatif vers l'url au lieu de déclarer l'url absolu avec des paramètres à chaque fois, on peut déclarer l'url root et on ajoute les paramètres/request avec path	Given url 'https://jsonplaceholder.typicode.com/' And path 'pages/editpage.action?pageId=22649888' =====c'est la même chose que ===== Given url 'https://jsonplaceholder.typicode.com/'
param	le paramètre à transporter dans la requête syntaxe : param nom du paramètre = 'valeur du paramètre'	Given url 'https://jsonplaceholder.typicode.com/' And param pageId = '22649888' =====c'est la même chose que ===== Given url "https://jsonplaceholder.typicode.com/"
request	requête à poster vers le serveur en format JSON ou XML en fonction du contexte	Given url 'https://myhost.com/v1/cats' And request { name: 'Billie', type: 'LOL' } =====ou bien format XML ===== And request <cat><name>Billie</name><type>Ceciling</type></cat>
header	l'entête du service qui va accompagner la requête lors de l'appel du service syntaxe : header nom du header = 'valeur du header'	Given url 'https://myhost.com/v1/cats' And header Authorization = 'montoken' And request { name: 'Billie', type: 'LOL' }

method	la methode HTTP du service get, post, put, delete, patch, options, head, connect, trace	Given url 'https://jsonplaceholder.typicode.com/ /phx/login/authenticate' And request { "username": "testAAAA", "password": :"testPPP" } And method post Then status 200
status	vérifier que la statut de la réponse du serveur égale le statut attendu dans le paramètre (200, 400, 404, 500 ...) si le statut est différent, le test sera en échec et les étapes suivantes du tests ne seront pas exécutées syntaxe status numéro	Given url https://jsonplaceholder.typicode.com/ /phx/login/authenticate And request { "username": "testcontrib", "password": :"testcontrib" } And method post Then status 200
response \$	Après chaque appel du webservice, le corp de la réponse du serveur est stocké dans le mot response jusqu'à nouvel appel HTTP le mot clé response peut être remplacé par \$	Given url https://jsonplaceholder.typicode.com/ /phx/login/authenticate And request { "username": "testcontrib", "password": :"testcontrib" } And method post Then status 200 And print response
response Time	le temps de réponse du serveur à chaque nouvelle requête le responseTime sera réinitialisé en fonction du temps pris pour obtenir la réponse le temps retourné est en seconde	Given url https://jsonplaceholder.typicode.com/ /phx/login/authenticate And request { "username": "testAAA", "password": :"testPPP" } And method post Then status 200 And print responseTime

response Header	l'entête de la réponse	
response Cookie	les cookies de la réponse	
match	vérifier qu'une valeur est égale ou pas à une autre. On ne peut pas utiliser les comparateurs '<' ou '>' si le résultat obtenu est différent du résultat attendu le test est KO on utilise match pour comparer des valeur de type text généralement syntaxe: <pre>match var1 == var2</pre> <pre>match var1 != var2</pre> <pre>match var1 contains var2</pre>	And match response == 1 And match response != 'test' And match 10 == 10 And match 'test' contains 't'
assert	il a le même rôle que le mot 'match' sauf qu'on l'utilise pour la comparaison des valeur numérique syntaxe : <pre>assert var == var1</pre> <pre>assert var >= var1</pre> <pre>assert var <= var1</pre>	And 1 == 1 And 4 > 2
read()	lire un fichier / récupérer des données à partir d'un fichier json/csv/text ... syntaxe: <pre>read('nomfichier.extension')</pre> si le fichier est dans le même package que la feature <pre>read('nompacage/nomfichier.extension')</pre> si le fichier est dans un autre package que la feature	<pre>*def var = read('data.json')</pre> <pre>*def var1 = read('employee/listEmployee.json')</pre>

Java.type()	Appeler une méthode java à partir d'une classe java et l'utiliser dans le gherkin Syntaxe: Java.type('nom de la classe') si la classe est dans le même package que la feature Java.type('nomPackage.nomClasse') si la classe est dans un package différent de la feature	Supposant j'ai une classe qui s'appelle salaire et elle se trouve dans un package paie. Dans cette classe j'ai une méthode getSalaire qui affiche le salaire d'un employé avec son matricule. Given def salaire = <code>Java.type('paie.salaire')</code> And monSalaire = <code>new salaire().getSalaire('m21597')</code> And print assert monSalaire > 1000
get	recupérer une valeur à partir du JSON syntaxe : get variableJson.xpathJson	*def id = <code>get response.id[0]</code> je récupère le premier id qui se trouve dans le JSON
function	déclarer une fonction java script dans le gherkin	Given func = function(){ return get \$[*].id }
call	appeler une fonction javaScript syntaxe: call fonction	Given id = call func()
Contains	vérifier qu'une valeur est contenue ou pas à une autre	* assert response.data.first_name == 'Janet'